# SCALE FOR PROJECT CYBERSECURITY - VA
# WEB (/PROJECTS/CYBERSECURITY-VACCIN

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere.42malaga.com:vogsphere/intra-uuid-49b0e4cf-4da9-4
```

## Introduction

Please respect the following indications:
- Be polite, courteous, respectful and constructive during the process
evaluation. The well-being of the community depends on it.
- Identifies with the person or group evaluated the possible failures of the work presented. Take the
and debate
the problems you have identified.
- You must understand that not everyone understands the statements of the same
way, which means that your vision of the project or what it
requests may vary. Always keep an open mind and evaluate the way
as sincere as possible. The proper functioning of pedagogy is based on
a correct evaluation.

## Guidelines

- Do not evaluate more than what is listed in the cloned git repository of the
student or group.
- Make sure that the git repository belongs to the right project,
and that it has run
"git clone" on an empty repository.
- Make sure that no aliases allow any form of cheating and that will correct
he official delivery.
- In order to avoid surprises, verify with the evaluated the
any scripts used to facilitate the evaluation.
- If, as an evaluator, you have not carried out the project you are going to evaluate,
you must read the statement completely before you begin to evaluate.
- Use the available flags to indicate an empty delivery, a
program that does not work, a cheating problem...
In these situations, the evaluation will be considered finished and the mark will be
0, or -42 if you suspect of cheating. However, except in cases of cheating,
we encourage you to continue exchanging opinions about the work
delivered, even not finished, to identify the problems that have
led to failure and avoid repeating them in the future.
- Throughout the entire evaluation, no segfault or
no program stop (unexpected, premature or uncontrolled)
In those cases, the final grade will be 0. Use the corresponding flag.
You should never have to edit any files except the
configuration, if any. In case you want to modify
a file, you must explicitly explain why you need to edit it and
agree with the evaluated student before touching any
thing.

## Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/154908/en.subject.pdf)

## Mandatory part

**Check**

The repository contains the files requested by the subject, namely:

- A README.md file indicating how the program works and what must be installed to use it.
- A Makefile allowing the compilation of the program if the code used requires a compilation.
- A test suite to help with this evaluation.

With the help of the person being evaluated, you should be able to execute the program requested by the subject for the rest of the evaluation.

If there is an error, you must stop the evaluation.

Yes No

**Request detection**

You must run the program with and without parameters. The program must:

- If no data is indicated it makes the GET request.
- The application handle at least the GET and POST method.
- The type of request can be indicated directly (GET, POST, PUT, DELETE...).
- All exceptions are controlled.

With the help of the evaluator you have to check that the program really executes a request by using the included tests
If there is an error, you must stop the evaluation.

Yes No

**Battery of tests**

The test batteries comply with the following sections:

- There are tests for at least two different database engines.
- There are tests of at least two different types (UNION, error, Boolean...).

The person being evaluated must be able to explain the methods/engines chosen and the reason for this choice.

With the help of the assessor, check that several types of attack are possible on at least 2 different engines.
We are not asking for a result but for an execution of these attacks.

If there is an error, you must stop the evaluation.

Yes No

**Data extraction**

If injection is possible, at least the following data are extracted:

- Names of the databases.
- Names of the tables.
- Names of the columns.
- All the data in the database.

With the help of the evaluator you have to check that the program really executes a request by using the included tests.
You should be able to clearly read the payload used and the result with the requested information.

If there is an error, you must stop the evaluation.

Yes No

# Bonus part

*Evaluate the bonus part if, and only if, the mandatory part has been entirely and perfectly done, and management handles unexpected or bad usage. In case all the mandatory points were not passed defense, bonus points must be totally ignored.*

**Bonus**

The program provides:

- Wider range of database engines. (1 point per engine added)
- Wider range of SQL injection methods. (1 point per method added)
- A way to edit various parameters of the request, e.g. the User-Agent. (1 point max)

**Rate it from 0 (failed) through 5 (excellent)**

# Ratings

**Don’t forget to check the flag corresponding to the defense**

✔ Ok

Empty work | Invalid compilation | Cheat | Crash | Incor

Concerning situation | Leaks | Forbidden fur

# Conclusion

**Leave a comment on this evaluation ( 2048 chars max )**

**Finish evaluation**

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33)
Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2)
Privacy policy (https://profile.intra.42.fr/legal/terms/5)
General term of use of the site (https://profile.intra.42.fr/legal/terms/6)
Rules of pro (https://profile.intra.42